

Iverto Backend — Load & Stress Test Report

Date: 24 Sep 2026, 10:50 – 11:45 IST (main run) · re-run 19:38 – 19:47 IST (\$2.7)
Target: `https://api.iverto.ai/devhostel1` — Hostel **v2** API (`/v2/tenants/acme/...`) + Socket.IO gateway
Tenant: `acme` (test accounts from `Test/*.csv` : 500 students, 756 guardians, 18 staff)
Tool: k6 v2.2.0, run from one Windows laptop (single public IP)
Traffic: reads only. No passes, decisions, pings or other writes were sent.
Scope of this document: findings and recommended fixes. **Nothing has been fixed.**

1. Verdict

FAIL. In its current state the dev backend would not survive a normal hostel evening rush.

#	Test	Result	Headline
1	Pre-flight (log in every account)	⚠ Blocked	Login is capped at 10 per minute per IP . 1,274 accounts would take about 2 hours.
2	Smoke: 4 users, every role	X Fail (latency)	100% functional, but p95 = 4.2 s (SLO 800 ms) at 4 users
3	Load: 30 journeys/s (busy-hour target)	X Fail (collapse)	91% errors , backend stopped answering after ~105 s, Caddy 502 from ~195 s
4	Stress: fine ramp 0.8 → 8 journeys/s	X Knee found	Throughput caps at ~30–35 req/s (≈ 3 journeys/s). Above that, latency climbs to 15–20 s
5	Socket.IO: ramp to 200 connections	X Fail (auth)	100% rejected: 44/mobile {"message":"Authentication failed"}
6	Diagnostic: 8 concurrent /dashboard	X Fail	All 8 returned HTTP 500 after ~12 s
7	Evening re-run: smoke + light load at 2 journeys/s	⚠ Pass with staff failures	Students/guardians 0% errors, p95 410 ms overall. But 39% of /dashboard and the only <code>/roster</code> call returned 500 , even at this low rate

Beyond performance, the run also surfaced **two release blockers in the shipped app (1.2.2)** and **one security gap**. See §3.

2. What was measured

2.1 Smoke — 4 VUs (1 per role), 18 journeys, 154 requests

Metric	Value	SLO
Checks passed	154 / 154 (100%)	> 99% ✓
HTTP errors	0%	< 1% ✓
Latency p50 / p95 / p99	168 ms / 4.21 s / 6.62 s	p95 < 800 ms X, p99 < 2 s X

Every endpoint works for a single user. The p95 comes almost entirely from staff screens:

Endpoint (single user, no load)	Response time
GET /roster?limit=20	7.4 s
GET /dashboard	3.7 – 4.2 s
GET /groups?limit=100	3.0 s
everything else	0.1 – 0.7 s

2.2 Load — target 30 journeys/s ($\approx$ 250–300 req/s), 6.5 min

Metric	Value
Requests	35,202
Failed	90.92% (32,008)
p50 / p95	9.35 s / 30 s (timeout)
Dropped journeys	6,040 (k6 had 1,500 VUs all stuck waiting on the server)
Status mix of failures	0 (30 s client timeout) and 502 Bad Gateway from Caddy, plus a few 500

Timeline (15 s buckets):

Time	req/s served	p50	p95	errors
0–15 s	27	0.4 s	1.6 s	0%
15–30 s	29	3.8 s	4.9 s	0.7%
45–60 s	23	13.4 s	15.3 s	1.8%
90–105 s	30	25.3 s	27.1 s	0.2%
105–120 s	35	29.3 s	30 s	52%
120–195 s	~50	30 s	30 s	100% (timeouts)
195 s → end	—	—	—	100% (Caddy 502)

After the run, the dev API kept timing out, then returned 502, and only recovered **about 2.5 minutes after the load stopped**. Production (/hoste1) stayed up the whole time.

2.3 Stress — ramp 0.8 → 1.6 → 3.2 → 4.8 → 6.4 → 8 journeys/s, 9.5 min

Offered load	Served req/s	p50	p95	errors
0.8 – 1.6 j/s	10 – 17	100 ms	290 – 340 ms	0%
1.6 – 3.2 j/s	19 – 25	160 – 190 ms	520 – 910 ms	0.3 – 0.9%
3.2 – 4.8 j/s (the knee)	30 – 34	0.3 → 3.8 s	1.1 → 4.9 s	0.5 – 0.9%
4.8 – 6.4 j/s	~34 (flat)	5.5 → 9.5 s	7.2 → 13.2 s	0.3 – 0.8%
6.4 – 8 j/s	~30 (flat)	12 – 15 s	13 – 20 s	0.3 – 0.7%

Throughput stops at about 30–35 requests/s however much load is offered. Above that, requests just queue. Overall: 15,649 requests, 0.42% failed, p95 15.8 s.

Every one of the 66 HTTP 500s in this run came from staff endpoints: /dashboard (56 of 66 calls failed), /groups and /roster.

2.4 Endpoint-isolation sweep (one token, constant concurrency)

Endpoint	5 concurrent	20 concurrent	Max throughput
GET /app-config	35 req/s, p95 185 ms	— (40 conc: 194 req/s , p95 316 ms)	~200 req/s ✓
GET /me/permissions/summary	172 req/s, p95 141 ms	47 req/s, p95 179 ms	~170 req/s ✓
GET /sites	29 req/s, p95 234 ms	52 req/s, p95 450 ms	~50 req/s
GET /me/branding	34 req/s, p95 187 ms	44 req/s, p95 623 ms	~45 req/s
GET /me/student	11 req/s, p95 541 ms	14 req/s, p95 1.68 s	~14 req/s ✗
GET /dashboard	1 call = 4.2 s	8 concurrent → all 500 at ~12 s	< 1 req/s ✗

While those 8 /dashboard calls were failing, /app-config stayed at ~0.4 s. **The Node process is not blocked. The database is.**

2.5 Socket.IO — ramp to 200 connections on /devhostel/socket.io, namespace /mobile

- The engine.io handshake and websocket upgrade work (101 on 20,586 of 20,620 attempts).
- **Every** namespace CONNECT with a v2 access token was rejected: 44/mobile,{"message":"Authentication failed"}.
- The high attempt count comes from the test reconnecting immediately after each rejection. The real app backs off.

2.6 Auth / rate limiting

- POST /v2/auth/password/login: the 10th login inside ~5 s returned 429 "Too many attempts" with Retry-After: 60. It cleared after ~55 s. **Limit ≈ 10 logins per 60 s**, observed **across different accounts from one IP**.
- The limit applied to **every** account and role, including staff.
- POST /v2/auth/refresh is in the same public-auth rate class in the contract.
- Access tokens expire after **15 minutes**.
- Responses have no RateLimit-* headers.
- Reads are **not** throttled: 600 reads at 20/s on one token all returned 200.

2.7 Evening re-run: smoke and light load at 2 journeys/s (19:38 – 19:47 IST)

What happened before it: An earlier afternoon attempt was cut short on the client side. The laptop's network switched mid-run, and the new network intercepted HTTPS (certificate chain issued by an authority that is not trusted). After that, every request failed before it reached the server. Certificate checks were **not** disabled. The run was repeated once the laptop was back on the original network. None of the failures below come from the client side.

Token pool: 35 tokens (24 students, 6 guardians, 3 wardens, 2 admins), 0 × 429. parent.acm0001@example.com still fails with both passwords (§P1-7).

Smoke (4 VUs, 1 per role): 153 / 153 requests OK. /roster, which had returned 500 in the afternoon smoke run, now returns 200. Latency still fails: p95 **5.4 s** (SLO 800 ms). **The staff screens are slower than this morning with no other traffic:**

Endpoint (single user)	Morning	Evening
GET /roster	7.4 s	9.1 s (max 9.6 s)
GET /dashboard	3.7 – 4.2 s	5.5 s (max 6.6 s)
GET /groups	3.0 s	1.8 s

Light load (2 journeys/s for 2 min, ~17 req/s, well below the ~30 req/s knee):

Metric	Value	SLO
Requests	3,497 (389 journeys)	
Failed	0.28% (10)	< 1% ✓
p50 / p95 / p99	103 ms / 410 ms / ~870 ms	p95 < 800 ms ✓
Throughput over time	flat at 17–18 req/s, no drift	✓

Every one of the 10 failures was an **HTTP 500 from a staff endpoint**:

Endpoint	Calls	p50	p95	Errors
GET /roster	1	17.5 s	17.5 s	1 × 500 (100%)
GET /dashboard	23	7.8 s	14.2 s	9 × 500 (39%)
GET /groups	3	2.7 s	4.7 s	0
All student and guardian endpoints	3,470	≤ 500 ms	≤ 1.1 s	0

What this tells us:

- The student and guardian paths are healthy at 2 journeys/s. The server is not overloaded at this rate.
- Staff reports now fail even at **5% of the target rate**. It takes only 1–2 wardens/admins hitting /dashboard at the same time as background student traffic for the call to cross the ~12–15 s server timeout and return 500. Since this morning's run the threshold has dropped from "8 concurrent" (§2.4) to "about 2".
- The 500s appear at 12–17 s, which matches a statement or pool-acquire timeout, not a crash. The Node process kept serving cheap endpoints in ~100 ms throughout.
- Single-user staff latency got **worse** during the day (roster +23%, dashboard +30%). Either data grew (every test run writes sessions, audit rows, notifications...) or the query plans changed. Both point to the missing indexes and aggregates in P0-3 / P0-4.

Raw files: `results/smoke-v2.{csv,json}`, `results/load-v2.{csv,json}`, `results/load-v2.csv.analysis.json` (these overwrote the morning's smoke/load files; the morning's numbers are preserved in §2.1–2.2). Stress and Socket.IO were **not** re-run. The knee is already known (§2.3), and re-running would take the dev server down again for no new information.

3. What went wrong, and how to fix it

Ordered by impact. **P0** = blocks release or production use. **P1** = fix before scale. **P2** = housekeeping.

P0-1 · The shipped app (1.2.2) cannot log in

What: `app.json` / `.env` point at `https://api.iverto.ai/devhostel`, and `lib/api/endpoints.ts` keeps `auth.*` on v1, so the app calls `POST /devhostel/v1/mobile/auth/login`. That route returns **404**

(`/devhostel/v1/mobile/me` is also 404). The devhostel deployment is v2-only. Every user is stuck on the login screen.

Evidence:

```
POST /devhostel/v1/mobile/auth/login      → 404 {"code":"NOT_FOUND",
"instance":"/devhostel/v1/mobile/auth/login"}
POST /devhostel/v2/auth/password/login    → 200 (tenantCode "acme", clientType "tenant_mobile")
```

Fix (pick one):

1. **Move the app's login to v2:** `POST /v2/auth/password/login` with `{tenantCode, identifier, password, clientType:"tenant_mobile"}`, and refresh via `POST /v2/auth/refresh`. The app needs a tenant-code field (or a hard-coded `acme` for this build). Note: v2 does **not** accept roll numbers or phone numbers as `identifier`. Only email worked (`ACM26-0001` → 401). The login screen copy and validation must change too.
2. **Or** deploy the v1 `auth/*` routes on `/devhostel` alongside v2 until the migration is finished.
3. Add a CI smoke check: `curl -f $API_URL/v1/mobile/auth/login` (or `/v2/...`) against the base URL baked into each build **before** it is signed.

P0-2 · Realtime (Socket.IO) rejects v2 sessions: no live updates

What: The `/mobile` namespace auth middleware still validates **v1** tokens. v2 tokens are opaque strings (`beRCaFe3...`, not JWTs), and every one gets `Authentication failed`. Live pass status, warden queues and notifications will never push.

Fix:

- Update the socket gateway's `auth` handler to validate v2 sessions (the same session lookup `/v2/auth/session` uses: `token` → `session` → `principal/tenant`).
- Or add a short-lived socket-ticket endpoint (e.g. `POST /v2/realtime/ticket`) and have the app send that ticket in `auth: { token }`.
- Add a test that connects with a fresh v2 token and expects a `40/mobile` ack.

P0-3 · Backend throughput ceiling ≈ 30 req/s, collapse and slow recovery

What: Mixed traffic saturates at 30–35 req/s. Past that, requests queue for 30 s, then Caddy returns `502`, and the service took ~2.5 min to recover after load stopped. A 500–2,000-student hostel opening the app before curfew easily exceeds this.

Why (evidence, not a guess):

- `/app-config` alone does ~**194 req/s** at p95 316 ms, so Caddy, TLS, networking and Node's event loop are fine.
- DB-backed endpoints plateau much lower (`/me/student` ~14 req/s). Heavy staff queries hold connections for 4–12 s.
- 8 concurrent `/dashboard` calls → all `500` at ~12 s while cheap endpoints stay fast.

→ **Database-bound: slow queries plus connection-pool exhaustion.** Everything queues behind a few heavy queries. The `502` phase means the upstream process died or was restarted (OOM or failed health check) under the queued backlog.

Fix, in order:

1. **Pull the server logs and metrics for 11:05–11:16 IST** (use the `requestId`s in the 500 bodies, e.g. `01a0d201-3422-760b-a72d-c6368ee545b4`). Confirm the pool-timeout / statement-timeout error, and check container restarts and memory (OOM kill).
2. **Profile the slow queries** with `EXPLAIN (ANALYZE, BUFFERS)` for `/dashboard`, `/roster`, `/groups`, `/me/student`, `/me/branding`, `/sites`. At 500 students these should be < 50 ms. Seconds means sequential scans or N+1 queries. Likely missing composite indexes: `(tenant_id, site_id)`, `(tenant_id, student_id)`, `(tenant_id, status, created_at)` on permissions, `(student_id)` on guardian links, `(group_id)` on group members.
3. **Remove N+1s**: `/me/student` embeds guardians, so fetch them with one join or `IN (...)` query, not one per guardian. Do the same for groups → members and the roster.
4. **Cache the dashboard**: compute counts with one aggregate query, or keep them in Redis / a materialized view refreshed every 15–30 s. It doesn't need to be real-time per request.
5. **Size and guard the pool**: set an explicit pool size (e.g. 20–30, below Postgres `max_connections`), a short acquire timeout (1–2 s), and a `statement_timeout` (~5 s). A saturated server should return a **fast 503 + Retry-After** rather than queue for 30 s.
6. **Give heavy staff reports their own small pool** (or a read replica) so a warden refreshing the dashboard can't starve students.
7. **Scale out** once the queries are fixed: run ≥ 2 app instances behind Caddy with `health_uri` active health checks and `lb_try_duration`, so one crashing instance doesn't mean 502 for everyone.
8. **Re-run k6/v2/stress.js** after each change. Target: a knee above 30 journeys/s (≈ 300 req/s).

P0-4 • `/dashboard` fails at 8 concurrent users, and its numbers don't add up

What: One call takes 4.2 s. Eight at once all return `500 INTERNAL_ERROR` at ~12 s. During the stress run, 85% of dashboard calls failed. Wardens and admins would see an error screen whenever a few of them open it together.

The payload is also inconsistent. `studentsInside.total` is 479 for the warden and 485 for the admin, while `inside 456 + outside 44 = 500`.

Evening update (\$2.7): it is now worse. Single-user `/dashboard` is 5.5 s and `/roster` 9.1 s. At only 2 journeys/s of mixed traffic, 39% of `/dashboard` calls and the only `/roster` call returned `500` at 12–17 s.

Fix: as in P0-3 (single aggregate query plus a cache). Concretely:

1. Run `EXPLAIN (ANALYZE, BUFFERS)` on the queries behind `/dashboard` and `/roster` first, since these two alone cause every 500 at low load.
2. Replace the per-request dashboard computation with one `GROUP BY` aggregate over the tenant/site, cached for 15–30 s (Redis or a materialized view). Target < 100 ms.
3. Paginate `/roster` in the database (`LIMIT/OFFSET` or keyset on an indexed column) instead of building the full list and slicing it. Add the `(tenant_id, site_id)` and student-status indexes it filters on. Target < 300 ms for `limit=20`.
4. Set a `statement_timeout` (~5 s) so a slow report returns a fast, retryable `503` instead of holding a pool connection for 15 s and starving other staff calls.

Also fix the counting logic so `total == inside + outside + unknown` for the same scope, and add a unit test asserting that invariant.

P1-5 • Login and refresh rate limit: 10/min per IP will lock out whole hostels

What: All students on hostel Wi-Fi share one public IP (NAT). With a 10/min per-IP limit on login **and** refresh (same `public-auth` class) and **15-minute access tokens**:

- 500 students each refresh 4× an hour ≈ **33 refreshes/min** from one IP, over 3× the limit. Refreshes get 429, the app treats that as a failed session, and users get logged out.
- After an app update or at the start of term, 500 logins from one IP take **50 minutes** to get through.
- Test tooling hits the same wall: verifying all 1,274 test accounts would take ~2 hours.

(I could only observe this from one IP. If the limit is **global** rather than per IP, the impact is even worse. Check the limiter config.)

Fix:

- Key **login** limits on *account + IP* (e.g. 5 failures per account per 15 min), not on IP alone.
- Move **refresh** out of `public-auth`, or give it a separate, far higher limit keyed on the refresh-token family.
- Allowlist known campus egress IPs, or raise the per-IP ceiling to thousands per minute for them.
- Send `RateLimit-Limit` / `RateLimit-Remaining` / `RateLimit-Reset` headers. In the app, treat 429 on refresh as "retry after N s", **never** as "log out".
- Consider a longer access-token TTL for mobile (e.g. 60 min) to cut refresh volume 4×.

P1-6 · Security: staff MFA is not enforced on MFA-level-1 routes

What: All staff logins (admins, wardens, security) return `mfaEnrollmentRequired: true`. The contract (`hostel-v2.bundle.yaml`) marks `/roster`, `/roles` and `/permissions` as `x-mfa-level: 1`. Yet those un-enrolled sessions got **200** with the full student roster (names, rooms, phones), role list and permission queue.

Fix: enforce `x-mfa-level` in the backend authorisation middleware: session `mfaLevel` must be ≥ the operation's required level, otherwise 403 with a problem type the app can route to enrollment. Until the app has an MFA enrollment screen, staff will then be locked out. Plan that app work alongside this fix.

P1-7 · Test-data / account problems

Problem	Evidence	Fix
student.acm0001@acme.edu gets a session that is instantly invalid	login 200, then every read 401 "Session invalid or expired". Other students work.	Check that student's principal/membership rows (possibly the reseed left an orphaned principal 04d99c70-...). Re-provision the account.
Two guardians share parent.acm0001@example.com	password 1 → 401 no active authority for tenant; password 2 → 401 invalid credentials	Email must be unique per guardian principal. Give the mother her own email and add a unique constraint.
25 students are graduated / inactive / suspended	students_credentials.csv	Expected, but exclude them from load pools (the scripts already do).
Junk guardian rows	roll numbers 123`, 1234, 123456, 1234576 linked to "Sajin"/"Suresh Gopi"	Delete or fix the leftover manual-test rows.
Roll-number / phone login rejected on v2	ACM26-0001 → 401; parent phone → not found (v1)	Decide whether v2 should resolve roll/phone identifiers; the old app login UX relied on them.

P2-8 · The original k6 suite is stale

`load-test/k6/*.js` targets `/hostel/v1/mobile/**` with per-VU logins. That path is gone on devhostel, and per-VU login is impossible under a 10/min limit. New v2 scripts were added for this run (see §5). Update `RUNBOOK.md` once the backend fixes land.

4. Capacity in plain terms

	Today (dev)	Needed
Sustainable mixed traffic	~25–30 req/s $\approx$ 2.5–3 app-opens/s	busy-hour target 30 journeys/s ($\approx$ 300 req/s)
Concurrent staff on dashboard	< 8 before errors	10–20 wardens/admins
Logins per minute (one campus IP)	10	hundreds
Realtime connections	0 (auth broken)	250–400 concurrent

About 3 app-opens per second covers a quiet period. It won't cover the pre-curfew rush, where hundreds of students open the app within a few minutes.

5. How the test was run (reproducible)

Files added (no app code was changed):

File	Purpose
load-test/data/accounts.json	1,274 accounts built from Test/*.csv (git-ignored)
load-test/scripts/verify-accounts-v2.mjs	v2 pre-flight: login plus one read per account
load-test/scripts/login-pool-v2.mjs	logs in a sample pool paced under the 10/min limit → data/tokens.json
load-test/k6/v2/{lib,smoke,load,stress,socket}.js	v2 journeys per role, mirroring lib/api/endpoints.ts
load-test/scripts/analyze-csv.mjs	per-endpoint and timeline stats from k6 CSV output
load-test/results/*-v2.*	raw CSVs, summaries, logs and analyses from this run

```
cd load-test
node scripts/login-pool-v2.mjs          # ~4 min, 35 tokens, valid 15 min
k6 run --out csv=results/smoke-v2.csv k6/v2/smoke.js
k6 run --out csv=results/load-v2.csv k6/v2/load.js          # RATE=30
k6 run -e PEAK=8 --out csv=results/stress-v2.csv k6/v2/stress.js
k6 run -e PEAK=200 k6/v2/socket.js
node scripts/analyze-csv.mjs results/stress-v2.csv 30
```

Role mix: student 80% · guardian 15% · warden 4% · admin 1%. Pool: 24 students, 6 guardians, 3 wardens, 2 admins, shared across VUs.

Caveats

- The generator was one laptop on one IP. The sweep proved it could push ~200 req/s, so the ~30 req/s ceiling belongs to the server, not the generator. The `dropped_iterations` in the load test come from VUs stuck behind the slow server.
- Because tokens were shared, per-account caching effects are slightly flattered. Real traffic from distinct users would be heavier on the DB, not lighter.
- Reads only. Write paths (submit, decisions, late entries) are untested and add DB load on top of this.

- Only 6 of 1,274 accounts were fully verified in pre-flight (rate limit). Of the accounts touched during the session, the failures were `acm0001` and `parent.acm0001` (§P1-7).
-

6. Suggested order of work

1. P0-1 app login → v2 (or deploy v1 auth on devhostel). **Without this, nothing else matters to users.**
2. P0-2 socket auth for v2 tokens.
3. P0-3 / P0-4: server logs → query profiling → indexes / N+1 / dashboard cache → pool + timeouts → second instance.
4. P1-5 rate-limit keys and refresh class.
5. P1-6 MFA enforcement (together with an app enrollment screen).
6. Clean test data (P1-7), then re-run smoke → load → stress. Pass criteria: p95 < 800 ms, errors < 1% at 30 journeys/s, socket connect failure < 5% at 400 connections.